

ADR-003-memory-provider-strategy

ADR-003: Memory Provider Strategy

Status

Accepted

Date

2026-01-08

Context

HelixCode requires long-term memory capabilities to enable AI agents to maintain context across sessions, learn from past interactions, and provide personalized assistance. The platform needs to support:

1. **Multiple memory backends:** Different use cases require different storage solutions
2. **Vector similarity search:** Semantic retrieval of relevant memories
3. **Metadata filtering:** Filter memories by project, user, time, tags
4. **Scalability:** Support from single-user to enterprise deployments
5. **Cost flexibility:** Balance between hosted services and self-hosted solutions
6. **Privacy compliance:** Support for on-premises deployments
7. **Integration flexibility:** Support both AI-native memory services and traditional vector databases

The challenge was creating a unified interface that supports diverse memory providers while enabling optimal provider selection based on use case requirements.

Decision

We implemented a unified `VectorProvider` interface with a provider registry and factory pattern, supporting multiple memory provider categories:

Provider Categories

AI-Native Memory Services (designed specifically for AI memory): - Mem0 (AI memory layer) - Zep (LLM memory server) - Memonto (persistent memory) - BaseAI (memory infrastructure)

Vector Databases (general-purpose similarity search): - Pinecone (managed vector DB) - Weaviate (open-source vector DB) - Qdrant (high-performance vector search) - ChromaDB (embedding database) - FAISS (Facebook AI Similarity Search) - Milvus (scalable vector DB)

Multi-Modal Platforms: - DeepLake (multi-modal vector store) - LlamaIndex (data framework)

Core Interface

```
type VectorProvider interface {
    // Core operations
    Store(ctx context.Context, vectors []*VectorData) error
    Retrieve(ctx context.Context, ids []string) ([]*VectorData,
        error)
    Update(ctx context.Context, id string, vector *VectorData)
        error
    Delete(ctx context.Context, ids []string) error

    // Search operations
    Search(ctx context.Context, query *VectorQuery)
        (*VectorSearchResult, error)
    FindSimilar(ctx context.Context, embedding []float64, k int,
        filters map[string]interface{})
        ([]*VectorSimilarityResult, error)
    BatchFindSimilar(ctx context.Context, queries [][]float64, k
        int) ([][]*VectorSimilarityResult, error)

    // Collection management
    CreateCollection(ctx context.Context, name string, config
        *CollectionConfig) error
    DeleteCollection(ctx context.Context, name string) error
    ListCollections(ctx context.Context) ([]*CollectionInfo,
        error)

    // Index management
    CreateIndex(ctx context.Context, collection string, config
        *IndexConfig) error
    DeleteIndex(ctx context.Context, collection, name string)
        error

    // Metadata operations
    AddMetadata(ctx context.Context, id string, metadata
        map[string]interface{}) error
    UpdateMetadata(ctx context.Context, id string, metadata
        map[string]interface{}) error

    // Management
    GetStats(ctx context.Context) (*ProviderStats, error)
    Optimize(ctx context.Context) error
    Backup(ctx context.Context, path string) error
    Restore(ctx context.Context, path string) error

    // Lifecycle
    Initialize(ctx context.Context, config interface{}) error
    Start(ctx context.Context) error
}
```

```

Stop(ctx context.Context) error
Health(ctx context.Context) (*HealthStatus, error)

// Metadata
GetName() string
GetType() string
GetCapabilities() []string
IsCloud() bool
GetCostInfo() *CostInfo
}

```

Data Model

```

type VectorData struct {
    ID          string          `json:"id"`
    Vector       []float64       `json:"vector"`
    Metadata     map[string]interface{} `json:"metadata"`
    Collection   string          `json:"collection"`
    Timestamp    time.Time       `json:"timestamp"`
    TTL          *time.Duration  `json:"ttl,omitempty"`
    Namespace    string          `json:"namespace,omitempty"`
}

```

Provider Factory

Providers are instantiated through a factory pattern:

```

func NewProvider(providerType ProviderType, config interface{})
    (VectorProvider, error)

```

Load Balancing

Support for multiple load balancing strategies: - Round-robin - Weighted (based on performance)
- Random

Cost Tracking

Each provider exposes cost information:

```

type CostInfo struct {
    Currency          string
    ComputeCost       float64
    TransferCost      float64
    StorageCost       float64
    TotalCost         float64
    BillingPeriod      string
    FreeTierUsed      float64
}

```

```
    FreeTierLimit float64
}
```

Consequences

Positive

1. **Provider Flexibility:** Easy to switch between memory providers
2. **Cost Optimization:** Can use cheaper providers for appropriate workloads
3. **Privacy Compliance:** Support for self-hosted providers (FAISS, Qdrant)
4. **Scalability:** From local FAISS to managed Pinecone
5. **Feature Access:** Each provider's unique features accessible through metadata
6. **Hybrid Deployments:** Can use multiple providers simultaneously
7. **Testing:** Mock providers enable comprehensive testing

Negative

1. **Feature Parity:** Not all providers support all features
2. **Configuration Complexity:** Each provider has different config requirements
3. **Migration Complexity:** Moving data between providers requires careful planning
4. **Performance Variance:** Different providers have different performance characteristics

Neutral

1. **Embedding Generation:** Currently delegated to LLM providers; may need tighter integration
2. **Memory Lifecycle:** TTL and cleanup policies vary by provider

Alternatives Considered

Alternative 1: Single Provider (Pinecone Only)

Description: Standardize on Pinecone as the sole memory backend.

Pros: - Simplified architecture - Consistent behavior - Good documentation - Managed service

Cons: - Vendor lock-in - Cost inflexibility - No self-hosted option - Internet dependency

Why Rejected: Enterprise customers require self-hosted options for data privacy. Cost sensitivity varies across deployments.

Alternative 2: Custom Vector Database

Description: Build a custom vector database optimized for AI memory.

Pros: - Full control over features - Optimized for HelixCode use cases - No external dependencies - Deep integration

Cons: - Significant development effort - Ongoing maintenance burden - Reinventing solved problems - Risk of suboptimal performance

Why Rejected: Existing vector databases are mature and well-optimized. Development effort better spent on core HelixCode features.

Alternative 3: PostgreSQL with pgvector

Description: Use PostgreSQL's pgvector extension for all vector storage.

Pros: - Single database for all data - Familiar technology - ACID transactions - Good enough performance for many use cases

Cons: - Limited scalability for large vector sets - No specialized memory features - Metadata flexibility limited - Not optimized for similarity search

Why Rejected: While pgvector works for small deployments, enterprise scale requires dedicated vector solutions. The interface allows pgvector as one option while supporting more scalable alternatives.

Alternative 4: LangChain Memory Module

Description: Use LangChain's memory abstractions.

Pros: - Pre-built abstractions - Community ecosystem - Multiple provider support - Python-native

Cons: - Python dependency - Limited Go support - External dependency risk - May not support all providers

Why Rejected: HelixCode is Go-based; adding Python dependency creates operational complexity. Native Go implementation provides better performance and integration.

Implementation Notes

- Provider implementations in `internal/memory/providers/`
- Factory pattern for provider instantiation
- Provider registry maintains available providers
- Health checks run periodically for monitoring
- Cognee integration provides additional memory capabilities

Provider-Specific Features

Mem0

- Automatic memory organization
- Cross-session continuity
- Memory graphs

Zep

- Message history management
- Entity extraction
- Summarization

Pinecone

- Serverless scaling
- Metadata filtering
- Namespace isolation

FAISS

- Local-first operation
- No network dependency
- GPU acceleration support

Weaviate

- GraphQL API
- Hybrid search (vector + keyword)
- Multi-tenancy

Related Decisions

- ADR-001: LLM Provider Interface (similar provider pattern)
- ADR-006: Database Schema Design (memory metadata storage)

References

- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/memory/providers/types.go`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/memory/providers/factory.go`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/memory/providers/mem0_provider.go`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/memory/providers/zep_provider.go`
- `/run/media/milosvasic/DATA4TB/Projects/helix_code/helix_code/internal/memory/providers/faiss_provider.go`